

A Governance-First Agentic Method for Teaching and Implementing Shor’s Algorithm: From Modular Period Finding to Auditable Quantum Workflows

Alejandro Reynoso

June 18, 2026

Abstract

This paper presents a pedagogical and governance-first interpretation of Shor’s algorithm through an agentic workflow implemented in Google Colab. The central claim is that difficult quantum circuits become easier to understand, validate, and reuse when their logic is decomposed into specialized agents with explicit responsibilities, inputs, outputs, invariants, failure modes, and audit evidence. The notebook models control-register preparation, modular-function design, joint-register construction, target-register measurement, periodic-state inspection, Quantum Fourier Transform analysis, control-register sampling, period recovery, factor extraction, and final natural-language explanation as a governed sequence. The approach preserves the mathematics of order finding while making each transformation inspectable. It explains how reversible modular exponentiation correlates two registers, how measurement isolates a periodic coset state, how the Fourier transform converts spacing into frequency-domain probability mass, and how classical post-processing validates a period and extracts factors. The paper then extends the same control pattern to quantum-financial algorithms, including portfolio optimization, amplitude-estimation-based pricing and risk measurement, scenario aggregation, and hybrid quantum-classical decision systems. The resulting framework is intended for advanced students and practitioners who require conceptual clarity, mathematical discipline, and model-governance controls suitable for high-stakes financial computation.

Colab notebook: [Governance-First Agentic Shor Notebook](#)

Contributions. This paper makes four specific contributions. First, it recasts the educational implementation of Shor’s order-finding routine as a sequence of auditable computational claims rather than as a single opaque circuit. Second, it specifies agent-level contracts for inputs, outputs, invariants, and evidence, so that pedagogical explanation is tied to machine-checkable state. Third, it distinguishes simulation convenience from algorithmic substance: the notebook may inspect statevectors for teaching, but period recovery is constrained to use measurement evidence and arithmetic validation. Fourth, it maps the same architecture to finance, where quantum routines must satisfy standards of data lineage, model validation, benchmark comparison, uncertainty reporting, and independent review.

Scope. The implementation is an educational NumPy simulation for the canonical case $N = 15$, not a claim of scalable fault-tolerant performance or near-term financial production readiness. Its value is methodological: it demonstrates how quantum algorithmic reasoning can be made inspectable without diluting the underlying mathematics.

1 Introduction and Motivation

Quantum computing is often introduced through a chain of correct but compressed statements. A qubit can exist in superposition; amplitudes can be complex; phase matters; interference can amplify or cancel amplitudes; and quantum algorithms exploit these effects to solve selected problems more efficiently than known classical procedures. The difficulty is not usually that any one of these statements is incomprehensible. The difficulty is that learners are asked to cross too quickly from one idea to the next. They may understand a formula for a statevector without seeing why phase is operationally important, or they may understand a circuit diagram without seeing what information is stored in each register. Shor’s algorithm is a particularly strong example of this pedagogical problem. It is widely described as an algorithm that factors integers by finding a period with the Quantum Fourier Transform [1, 3, 2], but this summary hides the essential narrative. It does not immediately explain what is periodic, where the periodic structure is encoded, why two registers are required, why measurement appears in the middle of the process, why the Fourier transform does not itself output the period, and why the final answer is still obtained through classical arithmetic.

The methodology developed in the accompanying Colab notebook begins by restoring that narrative. The starting point is the modular function

$$f(x) = a^x \bmod N,$$

where N is the integer to be factored and a is chosen so that $\gcd(a, N) = 1$. The classical

observation is that the function is periodic. There exists a smallest positive integer r , called the order of a modulo N , such that

$$a^r \equiv 1 \pmod{N}.$$

For the standard educational case $N = 15$ and $a = 2$, the modular sequence is

$$1, 2, 4, 8, 1, 2, 4, 8, \dots,$$

so the period is $r = 4$. Shor's algorithm does not search directly for the factors. It changes the problem into the task of identifying this hidden order. Once a valid even period has been found, and provided that $a^{r/2} \not\equiv -1 \pmod{N}$, the factors can be recovered from

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N).$$

For the example,

$$\gcd(2^2 - 1, 15) = 3, \quad \gcd(2^2 + 1, 15) = 5.$$

This reduction from factorization to order finding is the logical foundation of the entire method.

The quantum part begins with a control register. If the register contains n qubits, its computational space has dimension

$$Q = 2^n.$$

Applying a Hadamard gate to every control qubit creates the equal superposition

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle.$$

This state represents the complete input domain coherently. It is important to avoid saying that the register contains many classical integers in the ordinary sense. It contains a quantum superposition whose basis labels correspond to those integers and whose amplitudes are all equal.

A second register is prepared to store the modular-function output. A reversible modular exponentiation circuit implements

$$|x\rangle|0\rangle \longrightarrow |x\rangle|a^x \bmod N\rangle.$$

The joint state becomes

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |a^x \bmod N\rangle.$$

This equation is one of the most important in the paper because it clarifies a common misunderstanding. The modular values are not used as amplitudes. They are encoded as basis-state labels in the target register. Each branch of the control superposition is paired with its own modular output. Because the same modular values recur periodically, several control states become correlated with the same target state.

The modular circuit therefore produces a structured joint state. In the $N = 15$, $a = 2$ example, the first few branches are

$$|0\rangle|1\rangle + |1\rangle|2\rangle + |2\rangle|4\rangle + |3\rangle|8\rangle + |4\rangle|1\rangle + \dots.$$

The first and second registers are generally entangled because the state cannot usually be written as a simple product of an independent control state and an independent target state. The target value depends on the control value.

Measurement now plays a constructive role. Suppose the second register is measured and the outcome is a particular modular value y . Measurement projects the joint state onto the subspace satisfying

$$f(x) = y.$$

If the measured result is 1, then the surviving control values are

$$x = 0, 4, 8, 12, \dots$$

After renormalization, the control register becomes

$$\frac{1}{\sqrt{M}} (|0\rangle + |4\rangle + |8\rangle + |12\rangle + \dots).$$

The periodic structure is now visible in the positions at which the control-register amplitudes are nonzero. If the measured target value had been 2, the surviving positions would have been shifted, but the spacing would still be four. Thus measurement reveals a periodic coset rather than the period directly.

At this stage, measuring the first register would be premature. It would return one surviving position at random and would not efficiently expose the spacing. The next step is

a basis change. The Quantum Fourier Transform on a Q -dimensional register is defined by

$$QFT_Q|x\rangle = \frac{1}{\sqrt{Q}} \sum_{k=0}^{Q-1} e^{2\pi i x k/Q} |k\rangle.$$

Applied to the periodic control state, it converts regular spacing in the computational domain into concentration in the frequency domain. The underlying mechanism is interference among complex amplitudes. Quantum amplitudes behave like wave-like quantities because they possess both magnitude and phase. When several amplitudes contribute to the same output basis state, their relative phases determine whether they reinforce or cancel.

For a periodic state

$$\frac{1}{\sqrt{M}} \sum_{m=0}^{M-1} |x_0 + mr\rangle,$$

the amplitude of a Fourier-basis output $|k\rangle$ contains the sum

$$\sum_{m=0}^{M-1} e^{2\pi i (x_0 + mr)k/Q}.$$

The factor involving x_0 changes only the overall phase of the sum. The constructive or destructive behavior is controlled by

$$e^{2\pi i m r k/Q}.$$

For most k , these terms point in different directions in the complex plane and cancel. When

$$\frac{r k}{Q}$$

is close to an integer, the phases align and the amplitudes reinforce. Therefore, the transformed probability distribution has peaks near

$$k \approx \frac{jQ}{r}.$$

The QFT does not produce a classical frequency value by itself. It produces a quantum state in which frequency-related basis states have high measurement probability.

The first register is then measured in the computational basis. A single run returns one classical frequency index k . This value is interpreted through

$$\frac{k}{Q} \approx \frac{j}{r}.$$

The period is not always visible as the denominator of the unreduced fraction. Classical continued-fraction methods or bounded rational approximation are used to infer candidate denominators. Each candidate is validated by checking

$$a^r \bmod N = 1.$$

If the candidate fails, another measurement sample is used. The final factors are then produced classically through greatest common divisors.

This sequence reveals that Shor’s algorithm is inherently hybrid. Classical logic chooses a , checks coprimality, selects register sizes, interprets frequency samples, verifies candidate periods, and extracts factors. Quantum computation prepares the superposition, evaluates modular exponentiation coherently, creates the register correlation, and transforms periodic support into frequency information. The notebook deliberately preserves this division instead of presenting the procedure as a single monolithic quantum circuit.

The pedagogical challenge is that a conventional circuit diagram compresses all these steps. The learner sees Hadamards, controlled modular multipliers, an inverse QFT, and measurements, but may not know what conceptual responsibility each block carries. The agentic notebook expands the circuit into a sequence of specialized roles. One agent prepares the control register. Another defines the modular function. Another constructs the joint register state. Another measures the target register. Another inspects the periodic state. Another applies the Fourier transform. Another samples the transformed control register. Another recovers and validates the period. Another extracts and validates factors. A final explanatory agent reads the audit trail and explains what happened.

The term “agent” is used as a software and governance abstraction. It does not imply that a quantum gate is autonomous or intelligent. It means that a complicated workflow is decomposed into bounded responsibilities with explicit interfaces. Each agent receives declared inputs, performs one transformation, produces declared outputs, applies validation rules, and records evidence. This makes the notebook easier to understand because the learner can focus on one conceptual transition at a time.

The same architecture also supports governance. A sophisticated computational result is trustworthy only if the path from assumptions to output can be inspected. The control register must be normalized. The modular map must be correct. The joint state must preserve probability. The post-measurement state must be renormalized. The Fourier transform must preserve normalization. Frequency measurements must be recorded rather than selectively reported. Candidate periods must be tested. Factors must be nontrivial and must multiply back to the original integer.

The governance-first design is particularly important in educational simulations because the correct answer is often known in advance. It would be easy to compute the period classically and then unconsciously leak that information into the quantum recovery step. The notebook avoids this by treating the classical period as a benchmark retained only for validation. The recovery agent receives measurement evidence, not the known period.

The methodology is motivated not only by education but also by future applications. Quantum algorithms are likely to be deployed as components inside larger hybrid decision systems. Their outputs may influence portfolio allocations, risk limits, pricing, collateral, fraud detection, and capital planning. In those settings, an unexplained circuit output will not be sufficient. Institutions will need documented assumptions, controlled inputs, benchmark comparisons, validation tests, uncertainty estimates, and human-readable reports.

Financial applications are especially demanding because model risk and governance are already established disciplines. A quantum portfolio optimizer must show how returns, covariance matrices, constraints, and penalties were encoded. A quantum amplitude-estimation routine must disclose how probabilities and payoffs were prepared. A quantum risk engine must explain the mapping from scenarios to amplitudes and from measured frequencies to estimated losses. Agentic decomposition creates a natural structure for these controls.

The notebook therefore serves two purposes. First, it is a pedagogical implementation of the core logic of Shor’s algorithm. Second, it is a prototype for governable hybrid quantum-classical workflows. The central insight is that discovery itself can be decomposed into controlled stages. The quantum computer does not merely return an answer. It generates structured evidence through state preparation, entanglement, measurement, interference, and sampling. Classical agents interpret and validate that evidence. The result is a process that is not only mathematically correct, but also teachable, auditable, and transferable.

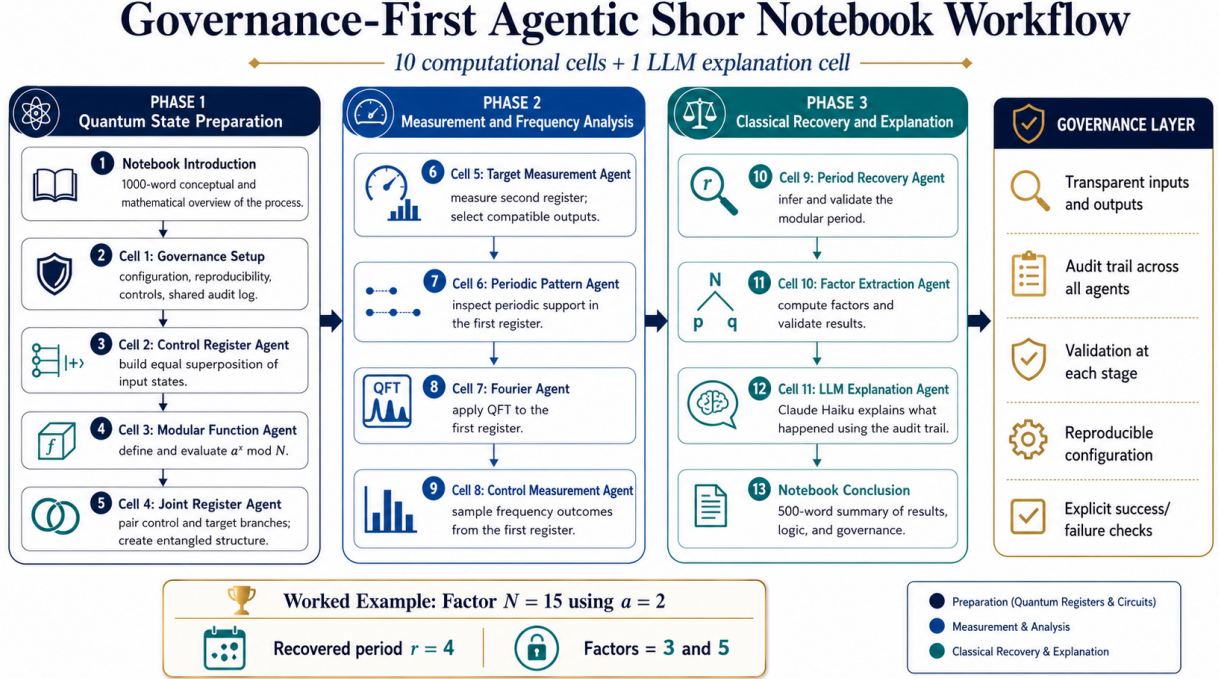


Figure 1: Governance-first agentic workflow of the Colab notebook.

2 Why Agentic: Simplifying Complex Circuits and Governing Discovery

The primary reason for using an agentic architecture is that quantum circuits are compact representations of processes that are cognitively dense. A small number of symbols can hide several distinct activities: state initialization, function definition, reversible arithmetic, entanglement, basis transformation, stochastic measurement, classical inference, and validation. For an expert, this compression is efficient. For a learner or a model-governance reviewer, it can be an obstacle. An agentic architecture expands the process into named responsibilities and makes the logical sequence visible.

An agent in this methodology is a bounded computational role. It has a specific mandate, a controlled input set, an output contract, validation rules, and an obligation to record evidence. The Control Register Agent is not allowed to alter the modular function. The Modular Function Agent is not allowed to infer the period from Fourier samples. The Period Recovery Agent is not allowed to read the benchmark period. This separation of duties reduces ambiguity and prevents circular reasoning.

The architecture also changes how discovery is represented. In a conventional notebook,

the successful result may appear as a continuous sequence of code. In the agentic notebook, discovery is treated as a chain of claims that must each be supported. The first claim is that the input register has been prepared correctly. The second is that the modular function has been computed correctly. The third is that the joint state represents the intended mapping. The fourth is that measurement has selected the correct subspace. The fifth is that the surviving support is periodic. The sixth is that the Fourier transform produces the expected concentration. The seventh is that measured frequencies imply a candidate period. The eighth is that the period satisfies the order condition. The ninth is that the factors are nontrivial and correct.

This stepwise structure is pedagogically powerful because each agent corresponds to a conceptual question. What is the control register? What is the target register? How is the modular function encoded? Why does entanglement appear? What does measurement do? What becomes periodic? Why is the QFT necessary? What is actually measured? How is the period inferred? Why are the factors produced classically? The notebook answers these questions in order.

The agentic design also exposes failure modes. Quantum algorithms are probabilistic, and not every run is informative. The target measurement may select any valid modular output. The Fourier measurement may return zero. The rational approximation may identify a denominator that divides the true period rather than equaling it. The candidate period may be odd. The factor extraction may produce a trivial factor. A governance-first agent does not hide these outcomes. It records them and either retries or stops according to explicit rules.

This is closely related to separation of duties in financial and operational governance. A model developer should not be the sole validator of the model. A data-preparation process should not silently change decision thresholds. A reporting layer should not rewrite numerical evidence. The notebook mirrors this discipline. The explanatory LLM can describe the audit trail, but it cannot alter the statevector, measurement counts, recovered period, or factors.

Agentic decomposition also makes the workflow reusable. The same pattern can be transferred to other quantum algorithms. In Grover search, one agent can define the oracle, another can apply phase marking, another can perform diffusion, another can measure, and another can validate the candidate. In amplitude estimation, agents can prepare the state, define the payoff operator, run phase estimation, decode the result, and construct confidence intervals. In QAOA, agents can define the Hamiltonian, initialize parameters, execute circuits, measure candidate solutions, update parameters, and validate constraints.

The architecture is therefore more than a teaching device. It is a general design pattern for hybrid quantum-classical systems. Quantum components become specialized services

inside a larger governed process. Classical agents manage data, parameters, validation, and reporting. Human reviewers can inspect the evidence at each boundary.

At a graduate level, the relevant question is not merely whether the notebook produces the known factors of 15. The stronger question is whether each computational claim has a well-defined object, a permissible information set, a falsifiable invariant, and a recorded failure mode. This is the difference between a demonstration and a governed computational experiment. Table 1 summarizes this standard.

Claim	Permitted evidence	Invariant or validation	Failure response
Control state prepared	Declared n , Q , statevector	$Q = 2^n$, $\sum_x \alpha_x ^2 = 1$	Stop or rebuild state
Modular map correct	N , a , evaluated values $a^x \bmod N$	$\gcd(a, N) = 1$, values in $\{0, \dots, N - 1\}$	Reject configuration
Joint state valid	Branch table $ x\rangle f(x)\rangle$ and amplitudes	Probability preserved under reversible mapping	Recompute branch table
Target collapse valid	Sampled target value and surviving support	Support satisfies $f(x) = y$, renormalized probability one	Resample or flag bug
Fourier evidence valid	QFT probabilities and measurement counts	Unitary normalization, counts retained without cherry-picking	Repeat measurement
Period accepted	Frequency sample, rational approximation, candidates	$a^r \equiv 1 \pmod{N}$ and candidate is usable	Test another candidate
Factors accepted	Valid period and GCD values	Nontrivial factors multiply to N	Retry with new evidence

Table 1: Graduate-level audit standard for the agentic Shor workflow.

The process also supports explainability. Quantum circuits are difficult to explain retrospectively if no intermediate evidence has been preserved. By contrast, an agentic notebook creates a narrative automatically because the audit trail records what each stage did. The explanatory agent can report that the target measurement selected a particular modular value, that the control support collapsed to specific positions, that the QFT concentrated probability at specific frequency indices, that a particular frequency sample implied a rational approximation, and that a candidate period passed or failed validation.

Finally, the agentic methodology makes governance part of computation rather than documentation added after the fact. Checks are executed at the moment when they are relevant. Normalization is checked after state preparation. Probability preservation is checked after joint-state construction. Candidate periods are tested before factor extraction. This creates a system in which correctness claims are supported by machine-readable evidence.

3 Governance-First Architecture of the Colab Notebook

The notebook begins with a governance configuration cell. It fixes the number N , the base a , the number of control qubits, the dimension Q , the random seed, the number of measurement shots, and a model identifier. These values are stored centrally rather than scattered across the notebook. The cell immediately checks that $Q = 2^n$ and that a is coprime to N . An invalid configuration cannot proceed silently.

A shared audit log is initialized before any state is created. Every later agent appends a structured record containing a timestamp, agent name, action, inputs, outputs, and checks. The audit trail is therefore a first-class output. It allows a reviewer to reconstruct the entire execution.

The architecture can be expressed as a controlled directed acyclic graph. Each node is an agent transformation

$$z_i = A_i(x_i; \theta_i),$$

where x_i is the permitted input evidence, θ_i is the declared local configuration, and z_i is the emitted artifact. Governance requires that each node also emits a check vector

$$c_i = C_i(x_i, z_i; \theta_i)$$

whose entries are Boolean validations or numerical diagnostics. Downstream agents receive artifacts and selected diagnostics, not hidden variables. In particular, the recovery stage must not receive the benchmark period computed for instructional comparison. This information-flow constraint is a core part of the method.

The Control Register Agent creates the equal-amplitude statevector

$$\frac{1}{\sqrt{Q}}(1, 1, \dots, 1)^\top.$$

It verifies that the squared magnitudes sum to one. The check is simple, but it establishes a pattern: every transformation is accompanied by an explicit invariant.

The Modular Function Agent evaluates $a^x \bmod N$ over the input range. It computes the classical order only as a benchmark. The benchmark is written to the audit log but is not passed to the recovery agent. This prevents answer leakage and preserves methodological integrity.

The Joint Register Agent builds an explicit list of branches $|x\rangle|f(x)\rangle$ and associates each with the control amplitude. In a full hardware implementation, this state would be

produced by a reversible modular exponentiation circuit, with qubit and circuit-resource designs studied in the Shor literature [4]. In the educational simulation, the branch table makes the relationship visible and auditable.

The Target Measurement Agent calculates the marginal probabilities of target-register outcomes. It then samples one target value using the governed random number generator. The corresponding control positions are selected and renormalized. The agent verifies normalization after collapse.

The Periodic Pattern Agent inspects the surviving positions and computes their spacings. This is a diagnostic step, not the official recovery mechanism. It helps the learner see that measurement has produced a periodically supported control state.

The Fourier Agent constructs the QFT matrix explicitly. This avoids hiding the basis transformation inside a black-box library routine. The agent multiplies the matrix by the control state, computes output probabilities, and checks normalization.

The Control Measurement Agent samples the Fourier probability distribution over multiple shots. It records the complete count distribution, including zero-frequency outcomes. The use of multiple shots reflects the probabilistic nature of the algorithm.

The Period Recovery Agent converts measured values into rational approximations. It generates candidate denominators and verifies each candidate through modular exponentiation. Failed candidates remain in the audit trail. A period is accepted only after the order condition is satisfied.

The Factor Extraction Agent checks that the period is even, computes the two greatest common divisors, rejects trivial factors, and verifies that the product equals N . The final governance report gathers configuration, measurement results, candidate records, validated period, factors, checks, and the complete audit log.

The eleventh code cell invokes Claude Haiku using an API key retrieved from Colab Secrets. The model receives only the governed evidence. It is asked to explain the process, distinguish simulation from hardware implementation, describe validations, and discuss failure modes. The LLM is a reporting agent, not a computational authority.

4 Workflow I: Register Preparation, Modular Evaluation, and Entanglement

The first operational stage is preparation of the control register. The notebook uses four control qubits, so $Q = 16$. Applying Hadamards to the all-zero state creates equal amplitude $1/4$ for each basis state. Every input value from zero to fifteen is represented coherently.

The target register begins in a known reference state. The modular exponentiation circuit then maps each control branch to its associated modular result. Conceptually,

$$|x\rangle|0\rangle \mapsto |x\rangle|2^x \bmod 15\rangle.$$

The first few branches form the sequence

$$|0\rangle|1\rangle, |1\rangle|2\rangle, |2\rangle|4\rangle, |3\rangle|8\rangle, |4\rangle|1\rangle.$$

The repeated target values encode the periodicity of the function.

The notebook refers to this step as joint-register construction because the modular circuit does more than simply “entangle” registers. It computes a specific reversible function. Entanglement is the structural consequence of that computation. The output cannot generally be written as a product of an independent control state and an independent target state.

This distinction matters for pedagogy. Saying that an agent “entangles the circuits” may suggest that entanglement is an arbitrary resource applied from outside. In fact, the relevant entanglement arises because the target register becomes conditionally dependent on the control value. The modular function determines the correlation.

The agentic design separates function definition from function execution. The Modular Function Agent is responsible for the mathematical map. The Joint Register Agent is responsible for representing its coherent action. In a larger implementation, these roles could be assigned to different software components: one generates a reversible circuit specification, while another compiles and executes it.

The notebook validates that total joint probability remains one. Because modular exponentiation is implemented as a reversible unitary transformation, normalization must be preserved. In the educational branch representation, this means the sum of squared branch amplitudes remains one.

This stage also makes clear why modular outputs are not statevector amplitudes. The amplitude attached to each joint branch remains $1/\sqrt{Q}$. The modular value appears in the basis label of the target register. Confusing values with amplitudes would destroy the structure of the algorithm and lead to incorrect interpretations of measurement.

The result of this stage is a coherent table of inputs and outputs. It resembles a classical function table, but all rows are represented in one quantum state. That state is not directly readable row by row. Its value comes from the correlations that later measurement and Fourier analysis exploit.

5 Workflow II: Measurement, Periodic Collapse, and Fourier Analysis

The target-register measurement is the conceptual bridge between modular evaluation and Fourier analysis. The second register is measured in the computational basis. If the outcome is y , all branches inconsistent with $f(x) = y$ are removed. The surviving branches are renormalized.

For $N = 15$ and $a = 2$, each modular output occurs at control positions separated by four. A measurement of 1 leaves positions 0, 4, 8, 12. A measurement of 2 leaves 1, 5, 9, 13. In each case, the offset differs but the period is identical.

The control register therefore collapses into a periodic coset state

$$\frac{1}{\sqrt{M}} \sum_m |x_0 + mr\rangle.$$

This state contains the hidden period in the spacing of its support. Measuring it immediately would reveal only one position. The QFT is required to convert the spacing into a probability pattern that can be sampled efficiently.

The Fourier transform is a change of basis from computational position to discrete frequency. Each basis state $|x\rangle$ is mapped to a uniform superposition whose phases rotate across frequency labels. When several periodically spaced basis states are transformed and added, their phases interfere.

The QFT output depends primarily on the period r , while the offset x_0 contributes a phase factor. This is why any target measurement outcome is useful: all possible cosets share the same frequency magnitudes.

For $Q = 16$ and $r = 4$, the transformed distribution has peaks at $k = 0, 4, 8, 12$. The spacing between peaks is $Q/r = 4$. A single measurement produces one of these indices according to the transformed probabilities.

The notebook uses multiple shots to make the distribution visible and to reduce dependence on one sample. This is pedagogically helpful and operationally realistic. Quantum algorithms produce samples, not direct access to the complete hidden statevector.

The QFT matrix is constructed explicitly in NumPy. Although a real quantum circuit implements the transform with Hadamards, controlled phase rotations, and swaps, the matrix representation makes the basis change transparent. The notebook checks that the transformation preserves normalization.

The measurement and QFT stages together demonstrate the relationship between phase and observability. Phase is not directly visible in a computational-basis measurement. The

Fourier transform mixes amplitudes so that phase relationships become probability differences. Measurement then reveals a frequency-related basis label.

6 Workflow III: Classical Period Recovery, Factor Extraction, and Explanation

After the QFT, the control register is measured. The resulting value k is interpreted through

$$\frac{k}{Q} \approx \frac{j}{r}.$$

The notebook does not assume that the denominator of k/Q is automatically the true period. The fraction may reduce, or the measured value may be uninformative.

The Period Recovery Agent applies rational approximation with a bounded denominator. It then generates candidate periods consistent with the observed denominator. Every candidate is checked through

$$a^r \bmod N = 1.$$

This verification step converts a probabilistic clue into a validated arithmetic statement.

If no valid period is found from one measurement, the agent moves to another informative sample. This reflects the real logic of the algorithm: repetition is sometimes necessary. The audit trail records the complete candidate history.

Once a valid period is found, the Factor Extraction Agent verifies that it is even. It computes

$$a^{r/2}$$

and then evaluates the two greatest common divisors. The resulting factors are accepted only if they are nontrivial and multiply to N .

The notebook therefore distinguishes evidence, inference, and validation. The QFT measurement is evidence. Rational approximation is inference. Modular-order checking is validation. Greatest-common-divisor computation is final extraction.

The explanatory LLM agent operates only after these steps are complete. It receives the configuration, measurement results, candidate periods, validation checks, factors, and audit log. It is instructed to explain the actual execution and not invent results.

This separation is crucial. An LLM is useful for pedagogy, documentation, and communication, but it should not be allowed to alter the quantitative record. The notebook treats the model as a narrator over governed evidence.

The final explanation can describe why the target measurement created a periodic state,

why the QFT produced peaks, why some measurements were uninformative, how the period was validated, and how the factors followed. It can also distinguish the NumPy simulation from a scalable fault-tolerant implementation.

The result is a complete chain from quantum state preparation to human-readable interpretation. Each stage is inspectable, and the final narrative is grounded in the recorded execution.

7 Applications to Quantum Computing Ideas in Financial Algorithms

The agentic governance pattern transfers naturally to quantum finance because financial algorithms are already hybrid, modular, and subject to oversight. A quantum-financial workflow typically begins with classical market data, transforms that data into a quantum representation, executes a circuit, samples outputs, and then returns to classical interpretation. This path contains the same classes of risk as the Shor workflow: data leakage, invalid encodings, probability nonconservation, uninformative samples, unsupported inference, and overconfident narrative reporting.

The relevant standard for finance is therefore stronger than “the circuit ran.” A governed quantum-financial routine should specify the financial quantity being estimated, the data vintage, the stochastic model, the encoding map, the circuit family, the optimizer or estimator, the sampling budget, the confidence procedure, the classical benchmark, and the conditions under which the output must be rejected. This is consistent with current model-risk principles: a model is not only a formula or program, but a decision process whose assumptions, limitations, validation evidence, and use restrictions must be documented [11].

Portfolio optimization provides a concrete example. A classical mean-variance formulation begins with expected returns μ , covariance matrix Σ , and decision vector w [5]. A constrained objective may be written as

$$\max_w \mu^\top w - \lambda w^\top \Sigma w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \in \mathcal{W}.$$

If the decision variables are discretized into binary holdings z , the problem can be transformed into a quadratic unconstrained binary optimization problem,

$$\min_{z \in \{0,1\}^m} z^\top Q z + q^\top z + \rho g(z)^2,$$

where $g(z)$ captures budget, cardinality, sector, leverage, or turnover constraints and ρ is a

penalty coefficient. An Encoding Agent must record the discretization grid, penalty scaling, constraint relaxation, and mapping from financial weights to qubits. A Circuit Agent may use QAOA or another variational ansatz [7, 8]. A Measurement Agent returns candidate bitstrings, not a guaranteed optimum. A Decoding Agent maps bitstrings back to weights, and a Validation Agent tests budget, exposure, concentration, turnover, and realized risk constraints.

This decomposition is not cosmetic. If the candidate portfolio is poor, the cause may be stale price data, an indefinite or ill-conditioned covariance estimate, an overly small penalty coefficient, a discretization that excludes the true optimum, insufficient circuit depth, optimizer stagnation, hardware noise, or an invalid decoding rule. Without agent boundaries, these causes are observationally entangled. With agent boundaries, each hypothesis has an audit artifact.

Amplitude-estimation-based finance gives a second example. Many pricing and risk tasks reduce to estimating an expectation

$$\mathbb{E}[h(X)] = \sum_x p(x)h(x),$$

where X is a market, credit, insurance, or operational-risk scenario and h is a payoff or loss function. Quantum amplitude estimation can estimate probabilities or expected payoffs by encoding the distribution and payoff into amplitudes [6, 10, 9]. In a governed workflow, a State Preparation Agent records the distributional assumptions and approximation error; a Payoff Agent records clipping, scaling, discounting, and payoff smoothness; an Estimation Agent records oracle calls and sampling budget; a Confidence Agent maps phase or amplitude estimates into intervals; and a Benchmark Agent compares the estimate with classical Monte Carlo under the same discretization.

The governance requirements are especially important for tail quantities. Value at Risk and Expected Shortfall depend on rare events, dependence assumptions, and distributional tails; modern market-risk capital frameworks place particular emphasis on expected shortfall and liquidity horizons [12]. A quantum speedup in an amplitude subroutine would not remove the need to justify scenario generation, copula or factor dependence, calibration windows, stress overlays, liquidity horizons, and backtesting. A Risk Aggregation Agent should therefore maintain explicit artifacts for marginal loss distributions, dependence structure, scenario construction, loss encoding, tail-threshold selection, and sensitivity to stress assumptions. The output should include not only a point estimate but also uncertainty, benchmark error, and a statement of model limitations.

Hybrid variational algorithms particularly benefit from agentic control. A Classical

Optimizer Agent proposes parameters; a Quantum Execution Agent estimates expectation values; a Convergence Agent monitors stopping rules; a Robustness Agent tests alternative initializations; a Noise Agent evaluates hardware sensitivity; and a Governance Agent records the complete optimization path. This structure prevents the final result from being reported without the path that produced it. For finance, the path matters because the same nominal objective can yield different decisions under different data windows, penalty scales, random seeds, optimizers, and execution noise.

Finance use case	Quantum subroutine	Governance evidence	Classical benchmark
Portfolio construction	QUBO/Ising encoding, QAOA, variational search	Data vintage, covariance method, penalties, constraints, bitstring counts	Mixed-integer optimization, convex relaxation, heuristic search
Derivative pricing	Amplitude estimation over payoff states	Scenario grid, state-preparation error, payoff scaling, confidence interval	Monte Carlo, finite-difference or tree model
Credit and insurance loss	Amplitude estimation or scenario sampling	Default/loss model, dependence assumptions, tail diagnostics	Monte Carlo with variance reduction
Market-risk aggregation	Quantum or hybrid tail probability estimation	Stress scenarios, liquidity horizons, backtesting, sensitivity analysis	Historical simulation, filtered simulation, parametric VaR/ES
Fraud or anomaly scoring	Quantum kernels or variational classifiers	Feature lineage, encoding, validation split, fairness and drift metrics	Logistic, tree-based, kernel, and neural baselines

Table 2: Agentic governance controls for representative quantum-financial workflows.

The methodology is also applicable to quantum machine learning, but the governance burden is high. Separate agents should govern feature selection, data encoding, circuit architecture, training, measurement, validation, fairness, drift, and explainability. Claimed performance improvements must be tested against strong classical baselines, identical train–test splits, and leakage controls. A model that obtains an impressive accuracy through preprocessing leakage or small-sample selection bias has not demonstrated financial value, regardless of whether a quantum circuit appears in the pipeline.

The explanatory LLM has a carefully bounded role in all these applications. It can summarize the audit trail, explain binding constraints, describe sensitivity, and communicate uncertainty. It should not substitute for quantitative validation, independent review, regulatory documentation, or investment judgment. In high-stakes finance, natural language is an interface to evidence, not a replacement for evidence.

The wider lesson is that quantum financial algorithms should be designed as governed decision systems. Computational novelty is not sufficient. Inputs must be controlled, assumptions documented, transformations validated, outputs benchmarked, limitations disclosed,

and failure conditions explicit. Agentic decomposition supplies an architecture for doing so while preserving the hybrid quantum-classical structure of the algorithms themselves.

8 Limitations and Evaluation Criteria

The paper deliberately separates educational value from performance claims. The notebook demonstrates the logic of Shor’s algorithm on $N = 15$, where the full statevector can be displayed and checked. That visibility would not be available at cryptographic scale. A scalable implementation would require efficient reversible modular arithmetic, fault-tolerant gates, error correction, resource estimation, and hardware-level validation. The agentic framework does not remove these requirements; it organizes the evidence needed to reason about them.

The same distinction applies to finance. The paper does not claim that near-term quantum hardware can outperform mature classical optimization, simulation, or machine-learning systems in production finance. It claims that if quantum components are introduced into financial workflows, they should be embedded in a governance architecture that makes data, assumptions, encodings, samples, inference, benchmarks, and limitations inspectable.

Future evaluation should therefore use both algorithmic and governance metrics. Algorithmic metrics include circuit depth, qubit count, oracle calls, sampling complexity, estimator variance, approximation error, and benchmark gap. Governance metrics include reproducibility, audit completeness, information-flow separation, validation coverage, failure-mode reporting, sensitivity analysis, and quality of human-readable explanations. A workflow should be considered mature only when it performs adequately on both dimensions.

9 Conclusion

The governance-first agentic notebook offers a pedagogical bridge between the mathematical elegance of Shor’s algorithm and the operational reality of implementing, validating, and explaining it. The method does not simplify the quantum mechanics by removing essential ideas. It simplifies the learning process by assigning each idea to a visible computational stage.

The control register, modular function, target register, joint-state construction, measurement, periodic collapse, Fourier transformation, frequency sampling, rational reconstruction, and factor extraction are treated as separate but connected responsibilities. This decomposition makes the discovery process inspectable.

The methodology clarifies several common misconceptions. The modular outputs are not amplitudes. Entanglement is created by the conditional modular computation. Measuring the target register selects a periodic coset but does not reveal the period directly. The QFT does not output the period; it transforms the state so that frequency-related values become observable through measurement. The final factorization remains a hybrid quantum-classical process.

Governance adds practical value. Every agent has declared inputs and outputs. Every important invariant is checked. Measurement outcomes are retained. Candidate periods are validated. Trivial factors are rejected. The explanatory LLM is restricted to reporting over the audit trail.

This structure is valuable beyond Shor’s algorithm. It can be applied to Grover search, amplitude estimation, phase estimation, quantum simulation, QAOA, VQE, and quantum machine learning. In each case, complex quantum logic can be decomposed into preparation, transformation, measurement, interpretation, validation, and reporting.

The relevance to finance is direct. Quantum-financial models will need to satisfy standards of model risk, data governance, fiduciary responsibility, and regulatory oversight. An algorithm that cannot explain how inputs became outputs will be difficult to deploy, regardless of its theoretical speedup.

The Colab notebook therefore functions as both an educational implementation and a prototype for governable hybrid systems. It demonstrates that advanced quantum ideas can be taught as controlled processes of discovery. The learner can see what was prepared, what was encoded, what became entangled, what was measured, how interference changed probabilities, how evidence was interpreted, and how the final answer was verified.

The central lesson is methodological: quantum algorithms become more understandable and more governable when they are represented not as opaque monolithic circuits, but as sequences of explicit, auditable decisions. The agentic approach preserves mathematical rigor while making the path to the result visible.

References

- [1] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010.

- [3] A. Y. Kitaev, “Quantum measurements and the Abelian stabilizer problem,” arXiv:quant-ph/9511026, 1995.
- [4] S. Beauregard, “Circuit for Shor’s algorithm using $2n + 3$ qubits,” *Quantum Information & Computation*, vol. 3, no. 2, pp. 175–185, 2003.
- [5] H. Markowitz, “Portfolio selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [6] G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, “Quantum amplitude amplification and estimation,” *Contemporary Mathematics*, vol. 305, pp. 53–74, 2002.
- [7] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” arXiv:1411.4028, 2014.
- [8] N. Moll, P. Barkoutsos, L. S. Bishop, J. M. Chow, A. Cross, D. J. Egger, S. Filipp, A. Fuhrer, J. M. Gambetta, M. Ganzhorn, A. Kandala, A. Mezzacapo, P. Müller, W. Riess, G. Salis, J. Smolin, I. Tavernelli, and K. Temme, “Quantum optimization using variational algorithms on near-term quantum devices,” *Quantum Science and Technology*, vol. 3, no. 3, article 030503, 2018.
- [9] N. Stamatopoulos, D. J. Egger, Y. Sun, C. Zoufal, R. Iten, N. Shen, and S. Woerner, “Option pricing using quantum computers,” *Quantum*, vol. 4, article 291, 2020.
- [10] S. Woerner and D. J. Egger, “Quantum risk analysis,” *npj Quantum Information*, vol. 5, article 15, 2019.
- [11] Board of Governors of the Federal Reserve System, “Supervisory Guidance on Model Risk Management,” SR 26-2, April 2026.
- [12] Basel Committee on Banking Supervision, *Minimum Capital Requirements for Market Risk*, Bank for International Settlements, January 2019.